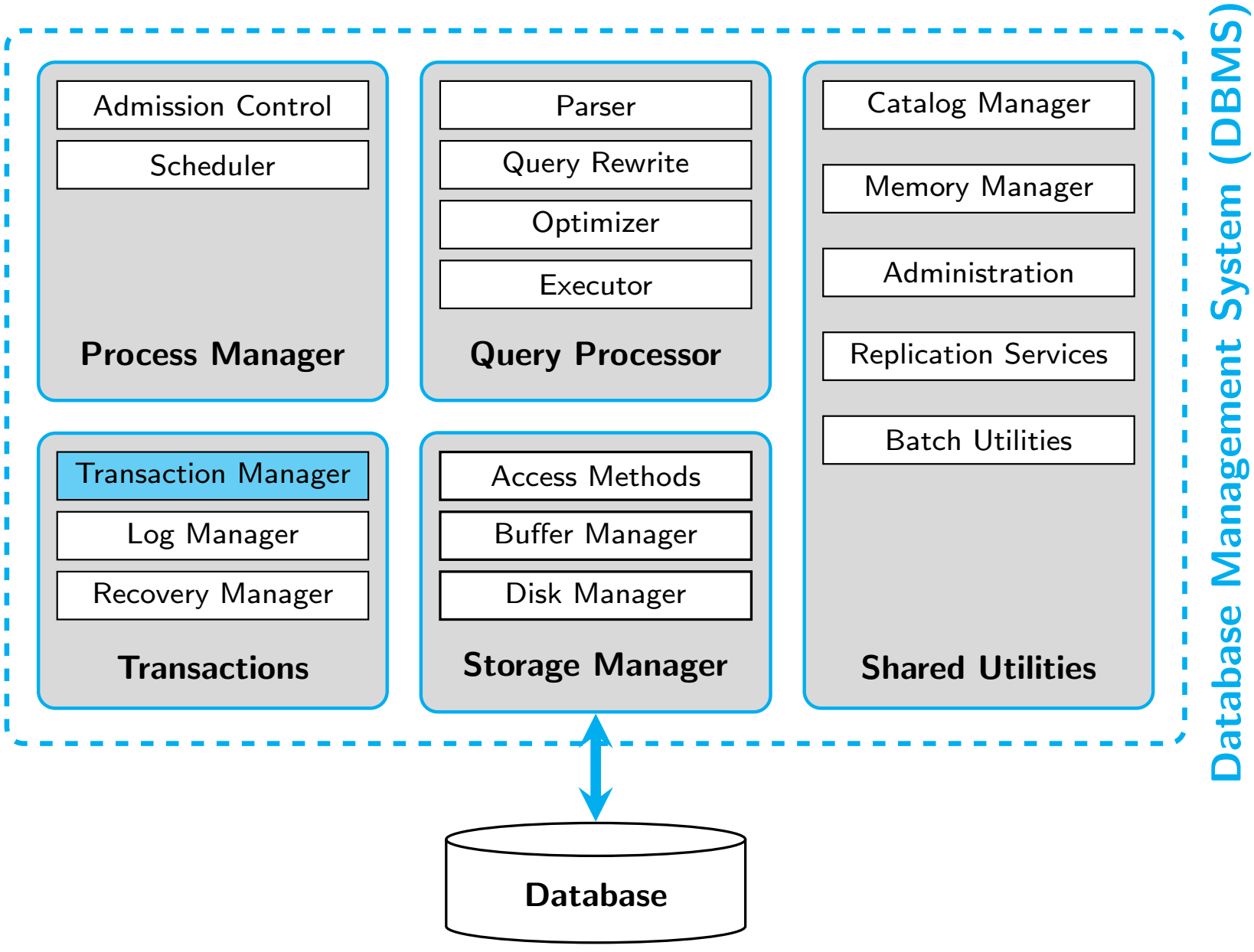
Control de concurrencia optimista

Clase 24

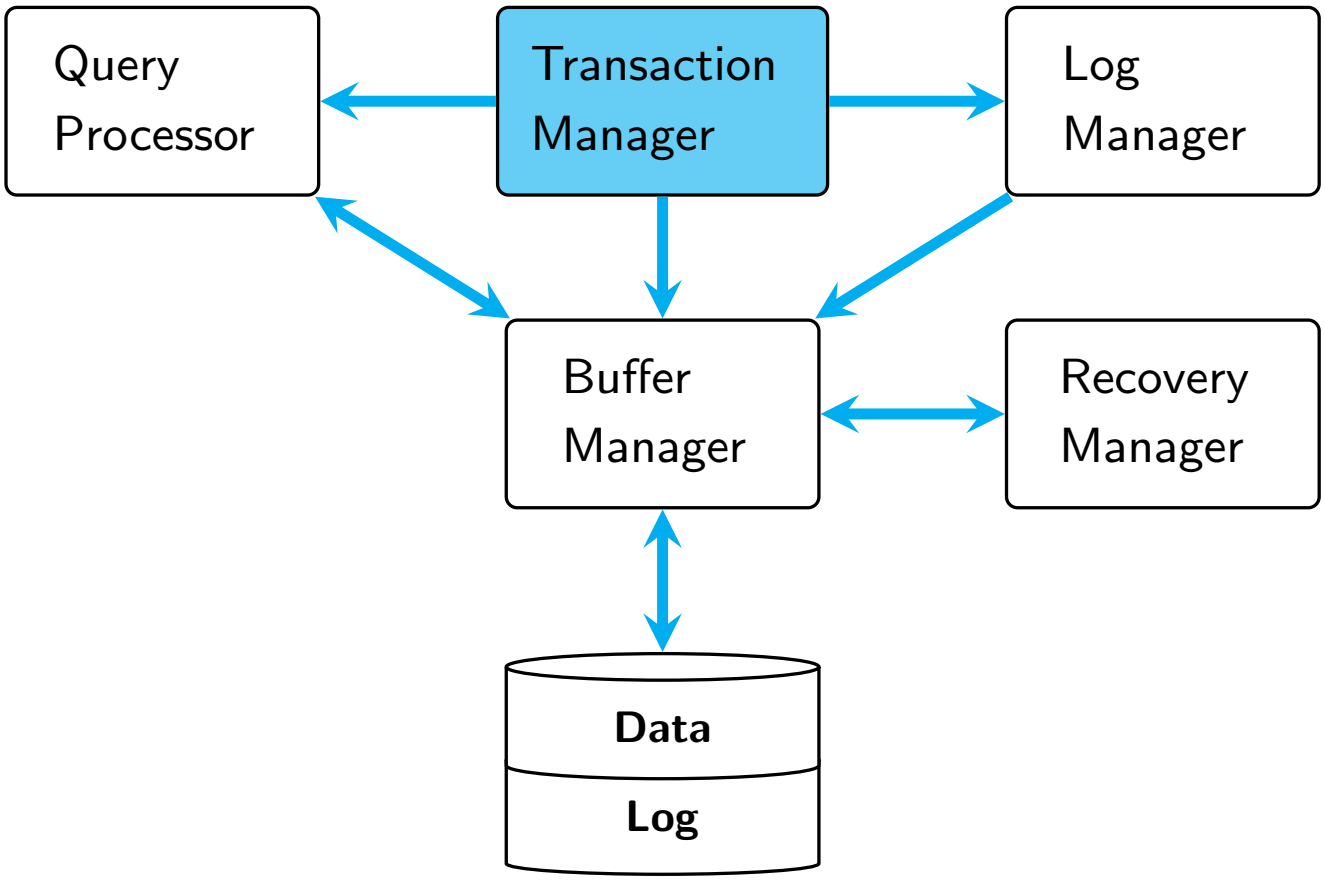
IIC2413 - Sección 3

Prof. Cristian Riveros

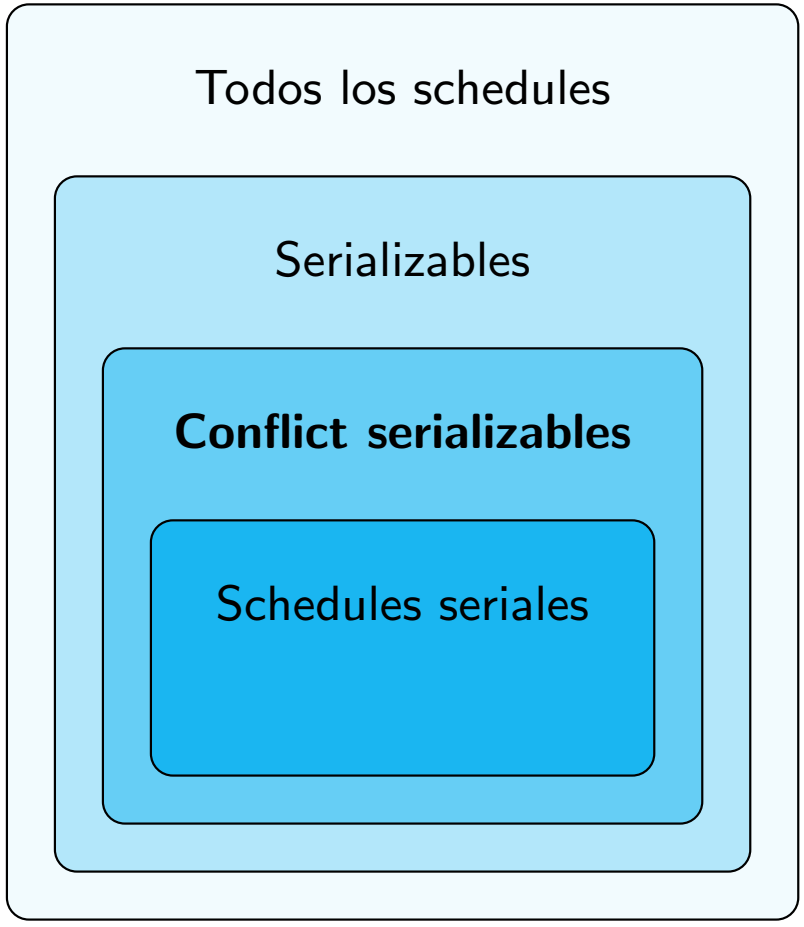
Manejo de transacciones



Zoom a la arquitectura de las transacciones



Relación entre tipos de schedules (recordatorio)



Conflict serializable es **más fácil de asegurar** que serializable

Outline

Locks (anterior)

Tuplas fantasmas (anterior)

Isolation en SQL (anterior)

Control de concurrencia optimista

Outline

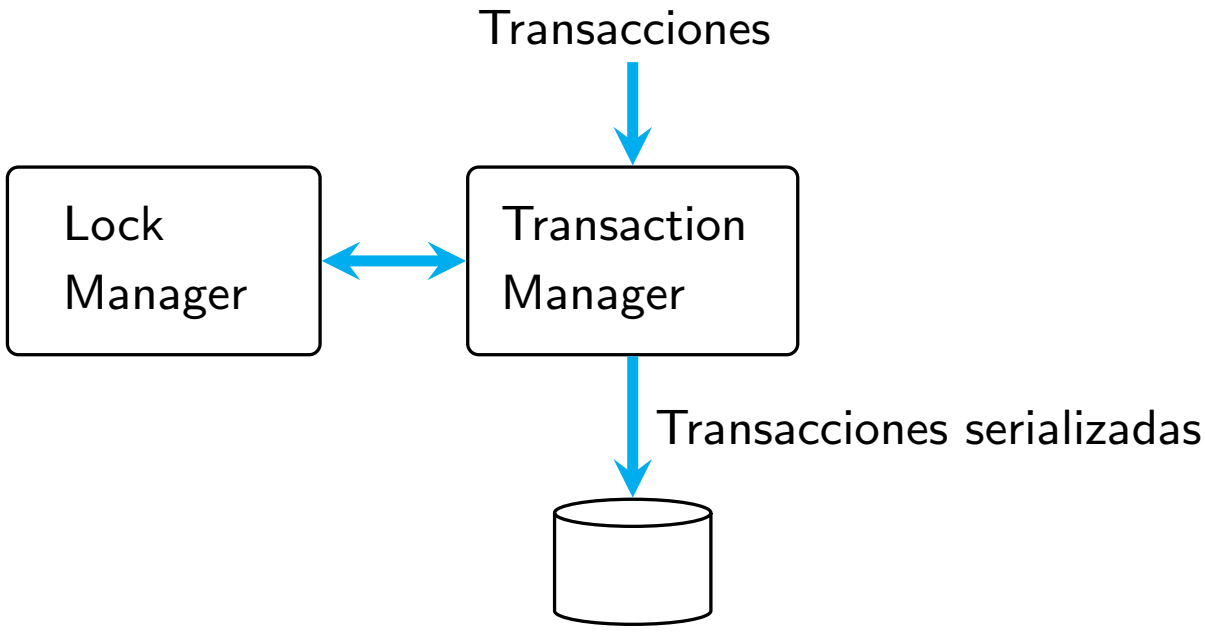
Locks (anterior)

Tuplas fantasmas (anterior)

Isolation en SQL (anterior)

Control de concurrencia optimista

Locks



Lock manager (o Lock table) interfaz:

- `requestLock`(*element*).
- `releaseLock`(*element*).

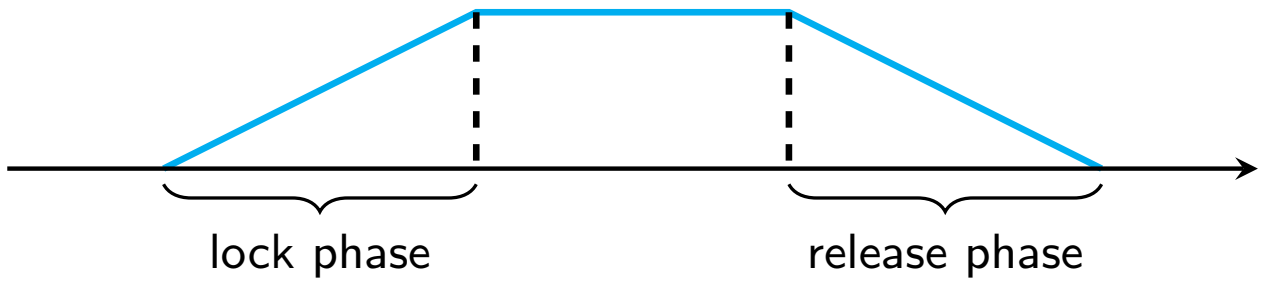
Principios de uso de locks

1. **Consistencia**:
   - Transacciones solo pueden leer o escribir un elemento si tienen su lock respectivo.
   - Si una transacción adquiere un lock, debe hacer released del lock en algún momento.
2. **Validez**:
   - solo una transacción puede tener el lock en cualquier momento.

Two-Phase Locking (2PL)

Toda transacción pasa por **dos fases**:

1. **Lock phase**: locks son pedidos a medida que son necesitados.
2. **Release phase**: Locks son liberados si ya no son requeridos.



"Una vez que la transacción libera algún lock,
no puede adquirir ningún lock nuevo."

Two-Phase Locking (2PL)

Ejemplo de ejecución con 2PL

$\mathbf{T_1}$	$\mathbf{T_2}$
$r_1(A)$	$r_2(A)$
$w_1(A)$	$w_2(A)$
$r_1(B)$	$r_2(B)$
$w_1(B)$	$w_2(B)$

$\mathbf{T_1}$	$\mathbf{T_2}$
lock A	
$r_1(A)$	
$w_1(A)$	
lock B	
unlock A	
	lock A
	$r_2(A)$
	$w_2(A)$
$r_1(B)$	
$w_1(B)$	
unlock B	
	lock B
	unlock A
	$r_2(B)$
	$w_2(B)$
	unlock B

Two-Phase Locking (2PL)

Ejemplo de ejecución con 2PL

$\mathbf{T_1}$	$\mathbf{T_2}$
$r_1(A)$	$r_2(A)$
$w_1(A)$	$w_2(A)$
$r_1(B)$	$r_2(B)$
$w_1(B)$	$w_2(B)$

$\mathbf{T_1}$	$\mathbf{T_2}$
$r_1(A)$	
$w_1(A)$	
	$r_2(A)$
	$w_2(A)$
$r_1(B)$	
$w_1(B)$	
	$r_2(B)$
	$w_2(B)$

¡el resultado es un schedule **conflict serializable**!

Two-Phase Locking (2PL)

Teorema

2PL produce solo schedules **conflict serializable**.

Demostración (idea)

Sea S un schedule como resultado del uso de 2PL.

- Para cada transacción T de S,
  sea u_T el tiempo en que T hace el primer unlock de un elemento.
- Ordene las transacciones de S como:

$$T_1, T_2, \ldots, T_n$$

  tal que $u_{T_1} < u_{T_2} < \ldots < u_{T_n}$.

PD: Para cada $i < j$, si $o_i(A)$ hace conflicto con $o_j(A)$ en S,
entonces $o_i(A)$ está antes que $o_j(A)$ en S.

Two-Phase Locking (2PL)

Teorema

2PL produce solo schedules **conflict serializable**.

Demostración (idea)

PD: Para cada $i < j$, si $o_i(A)$ hace conflicto con $o_j(A)$ en S,
entonces $o_i(A)$ está antes que $o_j(A)$ en S.

Por contradicción, suponga que existe una operación $o_i(A)$
que hace conflicto con una operación $o_j(A)$ tal que:

$$S = \ldots\ o_j(A)\ \ldots\ \mathbf{unlock}_j\, A\ \ldots\ \mathbf{lock}_i\, A\ \ldots\ o_i(A)\ \ldots$$

Como $u_{T_i} < u_{T_j}$, entonces el primer $\mathbf{unlock}_i$ de T_i esta antes que $\mathbf{unlock}_j\, A$.

¡contradicción! (¿por qué?)

Por la propiedad, el grafo de precedencia $\mathcal{G}_S$ de S es **acíclico**. □

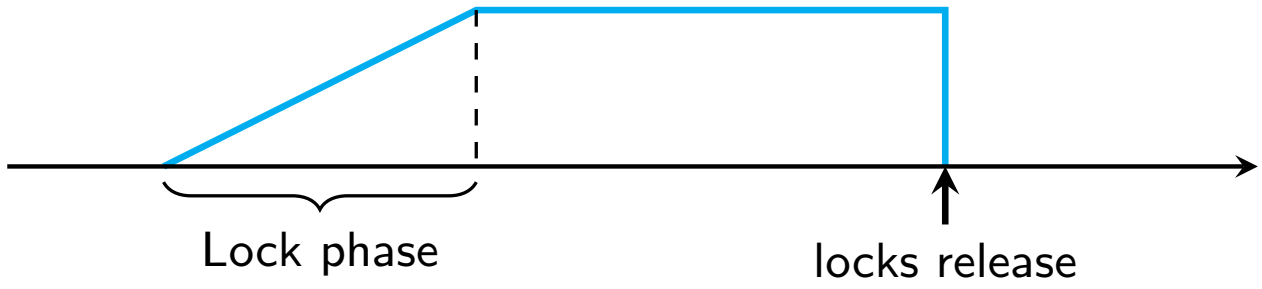
¿cuáles son los problemas con 2PL?

1. Cascading Rollback
2. Interferencia entre operaciones `READ`
3. Deadlocks

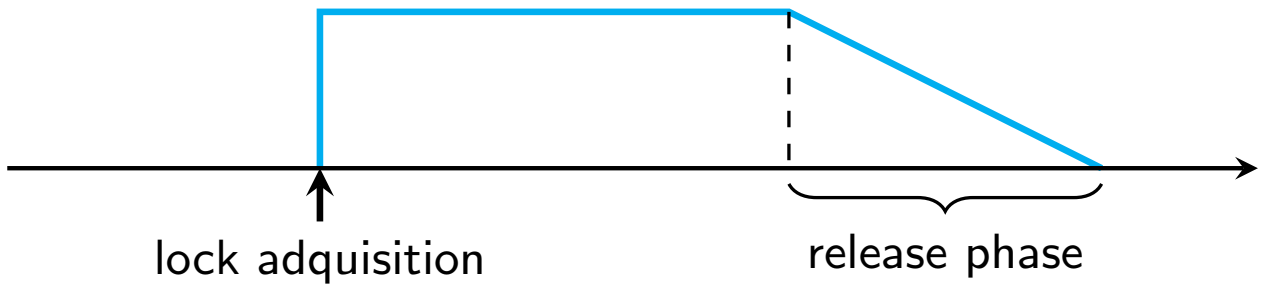
Cascading Rollback

Variaciones de Two-Phase Locking (2PL)

Strict 2PL.



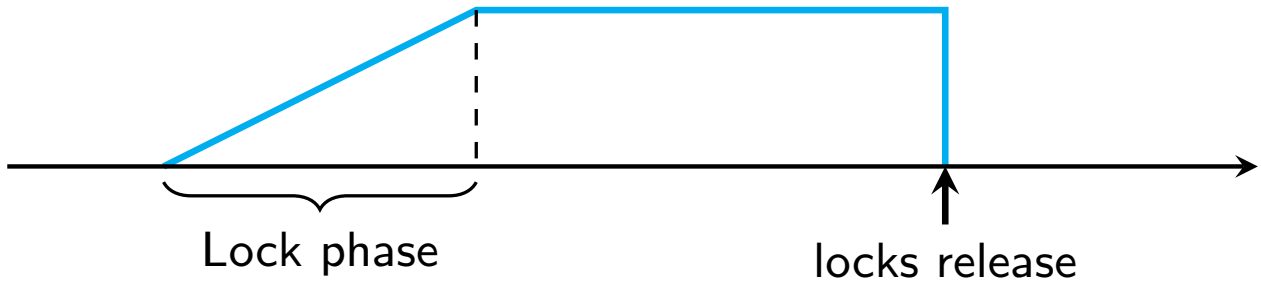
Preclaiming 2PL.



Cascading Rollback

Variaciones de Two-Phase Locking (2PL)

Strict 2PL.



Ventaja de Strict 2PL

- **NO** necesita de **Cascading Rollback**.
- Mas sencillo de implementar.

¿cuáles son los problemas con 2PL?

1. Cascading Rollback
   - Solución: Strict 2PL
2. Interferencia entre operaciones `READ`
3. Deadlocks

Interferencia entre operaciones `READ`

Tipos de lock modes

1. **Shared** locks (slock): para leer.
2. **Exclusive** locks (xlock): para escribir.

Reglas

- **slocks** pueden ser compartidos por varias transacciones.
- Una vez solicitado un **xlock** X de E, X debe esperar la liberación de todos los **slocks** o **xlocks** de E antes de ser otorgado.
- Una vez otorgado un **xlock** X de E, ningún **slock** o **xlock** de E puede ser otorgado hasta que el **xlock** X de E sea liberado.

Interferencia entre operaciones `READ`

Tipos de lock modes

1. **Shared** locks (slock): para leer.
2. **Exclusive** locks (xlock): para escribir.

Reglas

		Lock solicitado	
		S	X
Locks	S	Si	No
Actuales	X	No	No

¿cuáles son las **ventajas** de estos modos de locks?

Interferencia entre operaciones `READ`

Tipos de lock modes

1. **Shared** locks (slock): para leer.
2. **Exclusive** locks (xlock): para escribir.

Reglas

		Lock solicitado	
		S	X
Locks	S	Si	No
Actuales	X	No	No

Teorema

2PL + lock modes produce solo **conflict** serializable schedules.

¿cuáles son los problemas con 2PL?

1. Cascading Rollback
   - Solución: Strict 2PL
2. Interferencia entre operaciones `READ`
   - Solución: Shared y exclusive locks
3. Deadlocks



Deadlocks

Alternativas para evitar deadlocks

- Detección de ciclos en grafo de espera
- Timeout
- Prevención de deadlocks (timestamps):
  - wait-die
  - wound-wait

¿cuáles son los problemas con 2PL?

1. Cascading Rollback
   - Solución: Strict 2PL
2. Interferencia entre operaciones `READ`
   - Solución: Shared y exclusive locks
3. Deadlocks
   - Solución: detección/prevención de deadlocks

Outline

Locks (anterior)

Tuplas fantasmas (anterior)

Isolation en SQL (anterior)

Control de concurrencia optimista

Tuplas fantasmas / the phantom problem



Suposición

"Transacciones solo modifican: ¡no hay inserts o deletes!"

¿qué cambia si hacemos **inserts** o **deletes**?

Tuplas fantasmas / the phantom problem

Contraejemplo

	$\mathbf{T_1}$	$\mathbf{T_2}$	
Suma de valores $D_1, \ldots, D_n$ en y	$\texttt{READ}(D_1, x)$		
	$y := y + x$		
	$\vdots$		
	$\texttt{READ}(D_n, x)$		
	$y := y + x$		
		$\texttt{INSERT}(D_{n+1}, b)$	Insertar b y escribir en X
		$\texttt{WRITE}(X, c)$	
Escribir y en Y	$\texttt{WRITE}(Y, y)$		
Escribir en X	$\texttt{WRITE}(X, a)$		

¡este schedule **NO es serializable**!

Solución a tuplas fantasmas



Locks de mayor granularidad

- Atributo de una tupla
- Tupla/record
- Página
- Relación

Locking jerárquico (usado por DBMS comerciales)

Outline

Locks (anterior)

Tuplas fantasmas (anterior)

Isolation en SQL (anterior)

Control de concurrencia optimista

Anomalías de serialización (recordatorio)

1. Conflictos Write-Read: lecturas sucias (dirty reads)
2. Conflictos Read-Write: lecturas irrepetibles (unrepeatable read)
3. Conflictos Write-Write: actualización perdida (lost update)
4. Tuplas fantasmas

Niveles de Isolation en SQL

```
SET TRANSACTION ISOLATION LEVEL ****
```

1. SERIALIZABLE
2. REPEATABLE READ
3. READ COMMITTED
4. READ UNCOMMITTED

Niveles de Isolation en SQL

Nivel de Isolation	lecturas sucias	lecturas irrepetibles	actualización perdida	tuplas fantasmas
`SERIALIZABLE`	✗	✗	✗	✗
`REPEATABLE READ`	✗	✗	✗	✓
`READ COMMITTED`	✗	✓	✓	✓
`READ UNCOMMITTED`	✓	✓	✓	✓

1. `SERIALIZABLE`: Strict 2PL y locks jerárquicos.
2. `REPEATABLE READ`: Strict 2PL y locks en tuplas.
3. `READ COMMITTED`: Uso de slocks, pero son liberados inmediatamente.
4. `READ UNCOMMITTED`: Uso de xlocks pero no de slocks.

Utilidad de distinto nivel

1. `SERIALIZABLE`
   - Mayor nivel de isolation, menor performance.
2. `REPEATABLE READ`
   - Útil para transacciones sin `INSERT`/`DELETE`.
3. `READ COMMITTED`
   - Útil para transacciones con pocas modificaciones.
4. `READ UNCOMMITTED`
   - Útil para transacciones de solo lectura o tolerante a errores.

¿qué usan los sistemas clásicos?

Sistema	**Nivel por defecto**	**Nivel máximo**
Postgres	`READ COMMITTED`	`SERIALIZABLE`
MySQL	`REPEATABLE READ`	`SERIALIZABLE`
IBM DB2	`CURSOR STABILITY`	`REPEATABLE READ`
Oracle	`READ COMMITTED`	`SERIALIZABLE`
SQL Server	`READ COMMITTED`	`SERIALIZABLE`

(en DB2, `REPEATABLE READ` es equivalente a `SERIALIZABLE`)

OJO: los sistemas NO usan el máximo estándar **por default**

Outline

Locks (anterior)

Tuplas fantasmas (anterior)

Isolation en SQL (anterior)

Control de concurrencia optimista

¿qué es control de concurrencia optimista?



Observaciones

1. la mayoría de las transacciones son read-only.
2. la mayoría de las transacciones acceden datos disjuntos.

Suposiciones

- Comportamiento no-serializable es menor.
- Abortar transacciones si resulta un schedule no-serializable.

Esto se conoce como control de concurrencia **optimista**

Locking vs optimismo

- **Locking** previene comportamiento no-serializable.
- **Optimismo** permite ejecución hasta que schedule sea no-serializables.
  - En caso de llegar a un schedule no-serializable, usa `ROLLBACK`.
- **Locking** es eficiente para muchas transacciones de escritura.
- **Optimismo** es eficiente para muchas transacciones `READ-ONLY`.

Estrategia común: `Snapshot Isolation`

Snapshot Isolation



- Un tipo de control de concurrencia optimista y **multiversión**.
- Muy eficiente y popular en los sistemas comerciales.
  - PostgreSQL
- Previene la anomalías clásicas, pero aún así **NO** es serializable.

Snapshot Isolation: idea principal



Cada transacción utiliza
la **última versión disponible** de los datos cuando empezó

Cada transacción mira un **snapshot**
(**copia** instantánea)

Snapshot Isolation: formalización

Estructura

- Cada transacción **T** recibe un timestamp TS(**T**).
- Mantenemos multiversiones de un mismo elemento.

Reglas

1. Si **T** desea leer X, entonces **T** accede a la versión X_t donde:
   - t es un timestamp y
   - X_t es la última versión estable de X tal que $t < \text{TS}(\mathbf{T})$.
2. Si **T** desea escribir X, entonce funciona *First Commiter Wins*.
   - la transacción que pierde hace `ROLLBACK`.

Snapshot Isolation: formalización

Ejemplo

$\mathbf{T}_1$	$\mathbf{T}_2$	$\mathbf{T}_3$	$\mathbf{T}_4$
$r_1(X)$			
$w_1(X)$			
COMMIT			
	$w_2(X)$		
	ABORT		
		$r_3(X)$	
		$r_3(Y)$	
		$w_3(X)$	
			$r_4(X)$
			$r_4(Y)$
		$w_3(Y)$	
		$r_3(X)$	
		COMMIT	
			COMMIT

Snapshot Isolation: formalización

Ejemplo con versiones

$\mathbf{T}_1$	$\mathbf{T}_2$	$\mathbf{T}_3$	$\mathbf{T}_4$
$r_1(X_0)$			
$w_1(X_1)$			
COMMIT			
	$w_2(X_2)$		
	ABORT		
		$r_3(X_1)$	
		$r_3(Y_0)$	
		$w_3(X_3)$	
			$r_4(X_1)$
			$r_4(Y_0)$
		$w_3(Y_3)$	
		$r_3(X_3)$	
		COMMIT	
			COMMIT

Snapshot Isolation: formalización

Ejemplo *First Commiter Wins*

$\mathbf{T}_1$	$\mathbf{T}_2$	$\mathbf{T}_3$
$r_1(X_0)$		
$w_1(X_1)$		
`COMMIT`		
	$r_2(X_1)$	
	$w_2(X_2)$	
		$r_3(X_1)$
		$w_3(X_3)$
	`COMMIT`	
		`ROLLBACK`
		$r_3(X_2)$
		$w_3(X_3)$
		`COMMIT`

Snapshot Isolation

Ventajas

- Transacciones `READ-ONLY` nunca son retrasadas.
- Solo transacciones de escritura pueden ser abortadas.
- **Casi siempre** tiene un comportamiento serializable.

¿qué **anomalías de serialización** podrían ocurrir?

Anomalías comunes

1. Conflictos Write-Read: lecturas sucias (dirty reads) ✓
2. Conflictos Read-Write: lecturas irrepetibles (unrepeatable read) ✓
3. Conflictos Write-Write: actualización perdida (lost update) ✓
4. Tuplas fantasmas ✓

¿qué podría andar mal con **Snapshot Isolation**?

Anomalía Write-Skew

Ejemplo: $X + Y \geq 0$

$\mathbf{T}_1$	$\mathbf{T}_2$	X	Y
$r_1(X_0)$		100	100
$r_1(Y_0)$			
	$r_2(X_0)$		
	$r_2(Y_0)$		
$w_1(X_1)$		-100	
	$w_2(Y_2)$		-100
`COMMIT`			
	`COMMIT`		

Válido bajo Snapshot Isolation, pero **NO es serializable**

Snapshot Isolation: usuarios demasiado confiados

- Snapshot Isolation aparece como crítica al `ANSI SQL-92 standard`.

  A Critique of ANSI SQL Isolation Levels
  Berenson, Bernstein, Gray, Melton, O'Neil, and O'Neil, 1992.

- Implementado por la mayoría de los motores de bases de datos.
  - Ofrecido como `SNAPSHOT ISOLATION`.

OJO: usuarios pueden creer que es serializable

¿qué usa cada motor de base de datos?

- SQL Server:
  - `SERIALIZATION`: Strict 2PL
  - `SNAPSHOT ISOLATION`: MVCC
- MySQL:
  - `SERIALIZATION`: Strict 2PL
  - `REPEATABLE READ` y otros: MVCC
- IBM DB2: solo manejo de concurrencia basado en locks.
- PostgreSQL: solo manejo de concurrencia basado en MVCC avanzado.
- Oracle: solo manejo de concurrencia basado en MVCC.
  - `SERIALIZATION`: MVCC
  - No asegura anomalías de Write-Skew.

MVCC = Multi-Version Concurrency Control